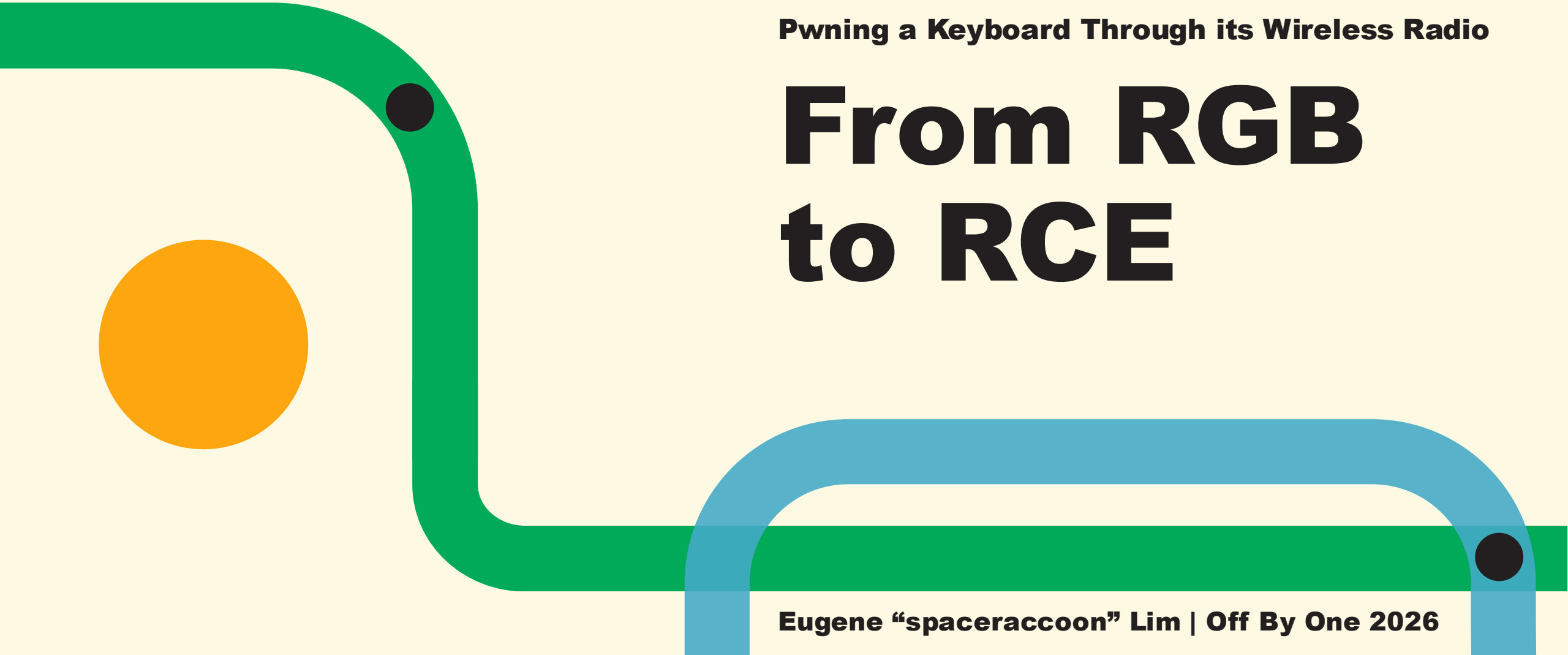


Pwning a Keyboard Through its Wireless Radio

From RGB to RCE

Eugene “spaceraccoon” Lim | Off By One 2026

Introduction

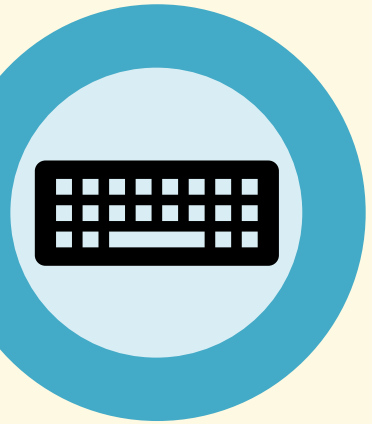
@spaceraccoon

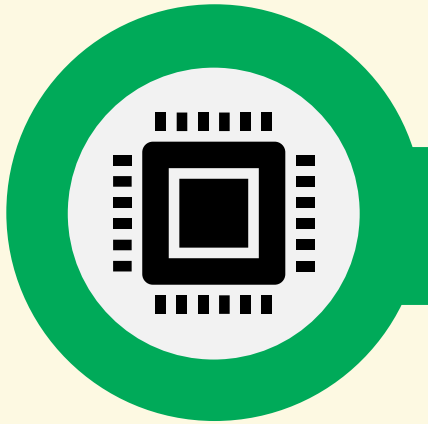
- Security researcher focused on enterprise software and consumer hardware
- Author of “From Day Zero to Zero Day”,
  No Starch Press (2025)
- Find me at spaceraccoon.dev



The Target

Product
Wireless keyboard





Hardware
ARM Cortex-M4F, nRF52 radio

Firmware
ZMK





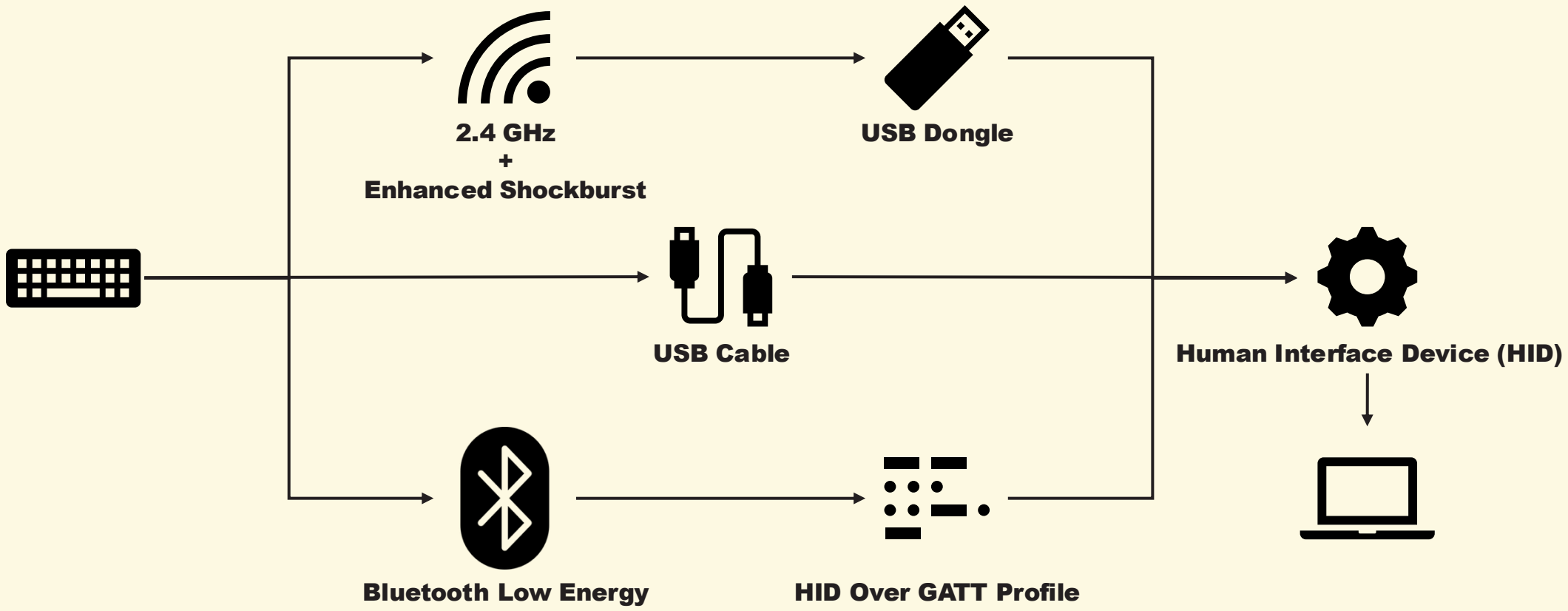
Protocols
HID, ESB, VIA

Agenda

1. Background
2. Wireless Decryption
3. Memory Corruption
4. ARM Shellcode
5. Takeaways

Background

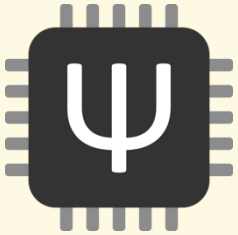
How do wireless keyboards work?



Modern keyboard firmware

QMK

- Optimized for wired keyboards
- Vendors add wireless support
- GPLv2 copyleft license (no bundling proprietary stacks)

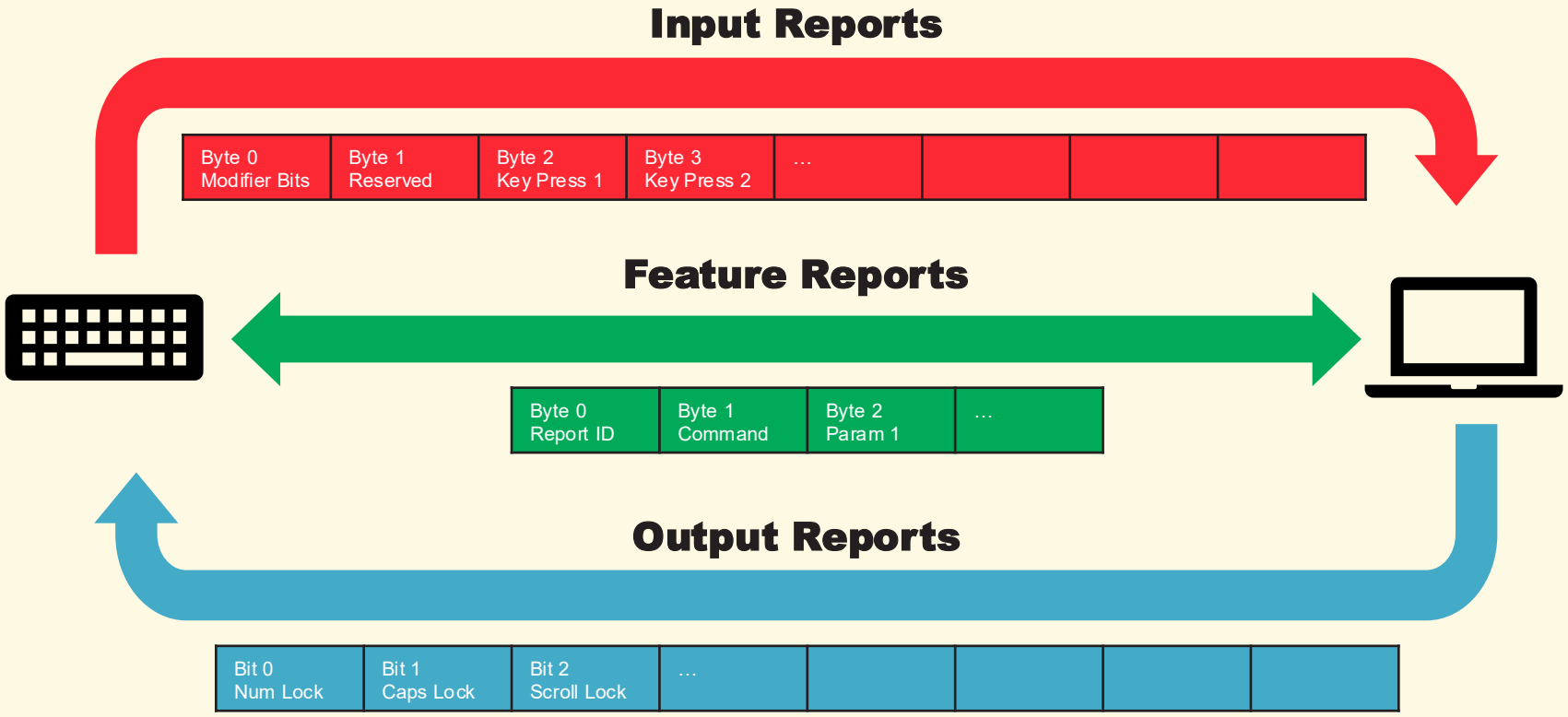


ZMK

- Native wireless support
- Built on Zephyr RTOS
- MIT license (bundling proprietary stacks OK)



Human Interface Device TL;DR



Human Interface Device TL;DR

zsh — zsh

```
$ hidutil list
Devices:
VendorID ProductID LocationID UsagePage Usage RegistryID  Transport Class                    Product
0x1050   0x406     0x3100000  61904     1     0x10000105d USB       AppleUserHIDDevice       YubiKey FIDO+CCID
$ ioreg -r -c IOHIDDevice -l
...
  +-o IOHIDInterface  <class IOHIDInterface, id 0x100001075, registered, matched, active, busy 0 (3 ms), retain 6>
      {
        "Transport" = "USB"
        "BootProtocol" = 0
        "HIDDefaultBehavior" = ""
        "Manufacturer" = "Yubico"
        "Product" = "YubiKey FIDO+CCID"
        "MaxInputReportSize" = 64
        "DeviceUsagePairs" = ({"DeviceUsagePage"=61904,"DeviceUsage"=1})
        "DebugState" = {"ReleasedBuffers"=0,"ReportAvailableCalls"=0,"ReportAvailableRuns"=0,"CreatedBuffers"=0}
        "MaxOutputReportSize" = 64
        "ReportDescriptor" = <06d0f10901a1010920150026ff0075089540810209211500 26ff00750895409102c0>
        "CountryCode" = 0
        "VendorID" = 4176
        "VersionNumber" = 1396
        "IOServiceDEXTEntitlements" = ("com.apple.developer.driverkit.transport.hid")
        "PrimaryUsage" = 1
        "LocationID" = 51380224
        "ProductID" = 1030
        "ReportInterval" = 2000
        "PrimaryUsagePage" = 61904
        "MaxFeatureReportSize" = 0
      }
```

38 FIDO Alliance Page (0xF1D0)

The FIDO (Fast IDentify Online) Alliance page provides usage definitions for devices that include Authentication features compliant with FIDO Alliance standards. The specification is available on the FIDO Alliance website[1].

Usage ID	Usage Name	Usage Types	Section
00	*Undefined*		
01	**U2F Authenticator Device**	CA	38.1
02-1F	*Reserved*		
20	Input Report Data	DV	38.1
21	Output Report Data	DV	38.1
22-FFFF	*Reserved*		

Table 38.1: Fast IDentity Online Alliance Page

PrimaryUsagePage = 61904 = 0xF1D0

MaxInputReportSize = 64

MaxOutputReportSize = 64

Viable Interrupt Alternative (VIA)



A keyboard configuration protocol that rides in raw 32-byte HID packets with a simple opcode and data structure.

ID	Command	Effect
0x01	get_protocol_version	handshake
0x02/0x03/0x04	get/set keyboard value	uptime, layout options, matrix state…
0x05	dynamic_keymap_set_keycode	write one (layer,row,col) → keycode
…		

Typically **restricted to USB cable inputs due to configuration powers…**

Enhanced Shockburst Protocol

A Layer 2 protocol developed by Nordic Semiconductor used by many 2.4GHz radios.



Preamble

Used for bit / clock sync

Address

Filters packets matching addresses used by the receiver; deconflicts packets on the same channel

Packet Control Field

Payload length (6 bits)
Packet ID (2 bits)
NO_ACK flag (1 bit)

Payload

Wrapped data e.g., HID

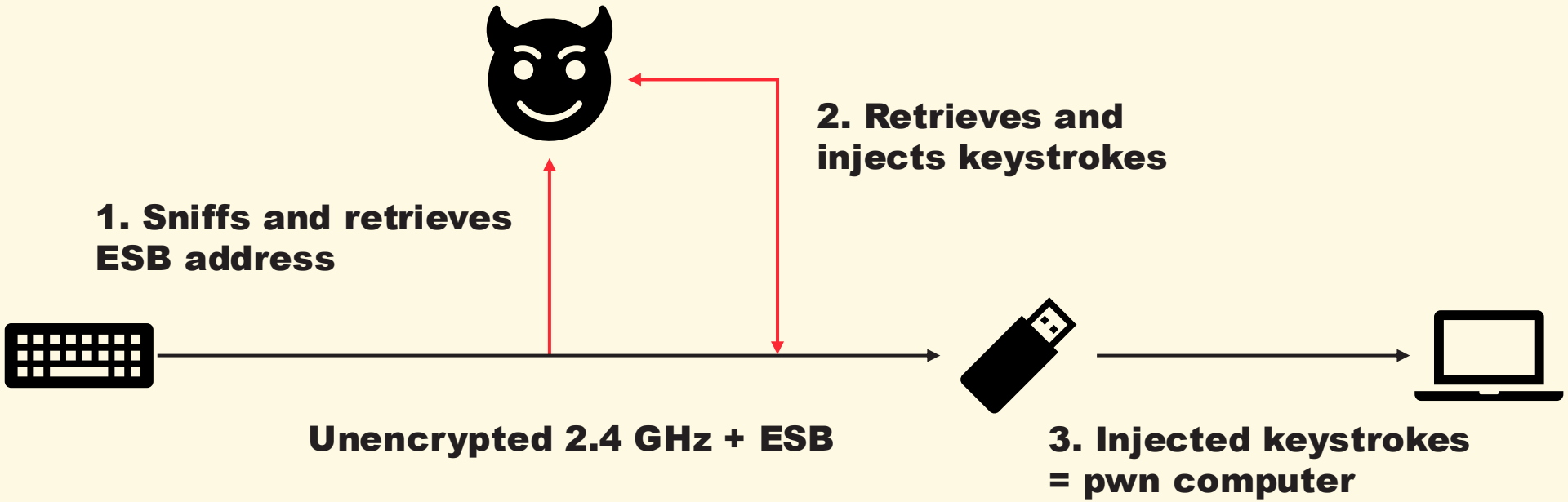
CRC

Calculated over the Address, PCF, and Payload

Wireless Decryption

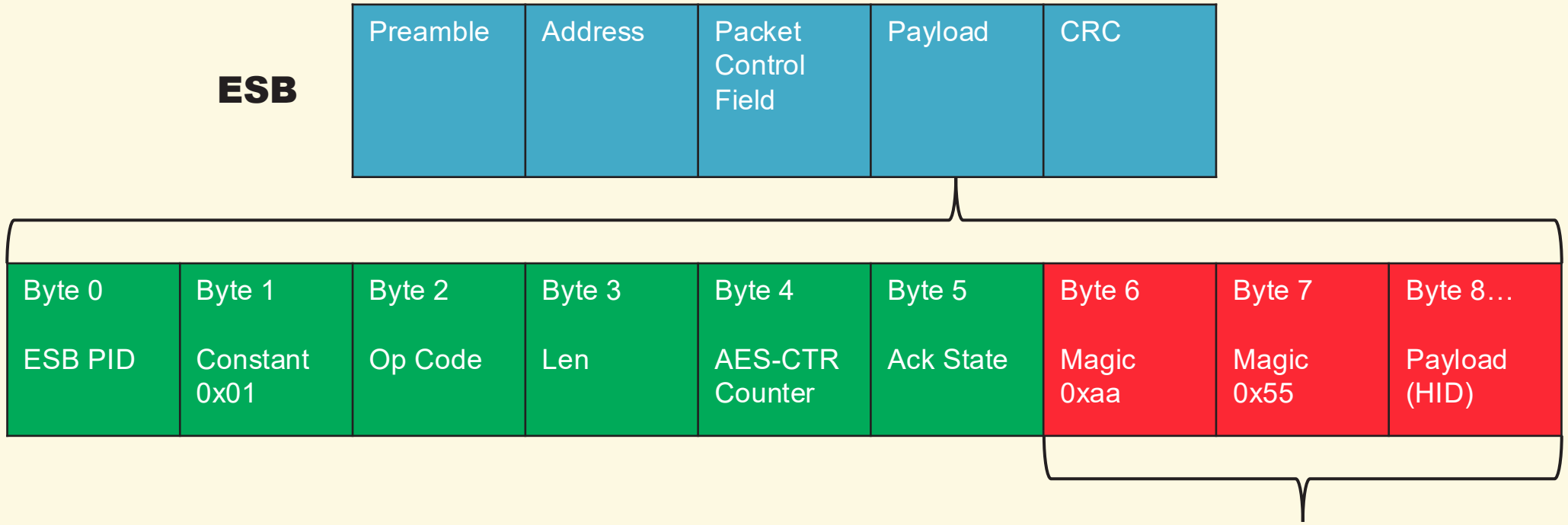
The MouseJack Attack

Publicized by Bastille's Threat Research Team in 2016, it exploited an era of missing encryption in many wireless peripherals.



Spoiler: This talk isn't about the MouseJack attack.

How the keyboard encrypts



Encrypted with an AES-128 keystream using a secret key – is this done with a key establishment protocol or proper crypto?

Weak crypto implementation

aes_ccm.c from zephr open-source ✅

```
// parameterized key, not hard coded
static int ccm_crypt(const uint8_t key[16], const uint8_t nonce[13], const
uint8_t *in_msg, uint8_t *out_msg, uint16_t msg_len)
{
    // ...
    // fresh keystream per-block
    for (j = 0; j < blk_cnt; j++) {
        /* S_1 = e(AppKey, 0x01 || nonce || 0x0001) */
        sys_put_be16(j + 1, &a_i[14]);
        err = bt_encrypt_be(key, a_i, s_i);
        // ...
        if (j < blk_cnt - 1) {
            xor16(&out_msg[j * 16], s_i, &in_msg[j * 16]);
        } else {
            for (i = 0; i < last_blk; i++) {
                out_msg[(j * 16) + i] = in_msg[(j * 16) + i] ^ s_i[i];
            }
        }
    }
    return 0;
}
```

decompiled encryption function from firmware ❌

```
void zmk_24g_enc_data(uint8_t *p_data,uint8_t len,uint8_t seq)
{
    byte *pbVar1;
    uint uVar2;
    uint8_t enc [16];

    aes_ctx[0xf] = seq << 1;
    // ...
    aes_ctx[0xe] = seq;
    // USES HARD CODED KEY
    ecb_encrypt_be(const_ek,aes_ctx,enc);
    pbVar1 = p_data + -1;
    // loops the keystream per-block instead of extending it
    for (uVar2 = 0; (int)uVar2 < (int)(uint)len; uVar2 = uVar2 + 1) {
        pbVar1 = pbVar1 + 1;
        *pbVar1 = enc[uVar2 & 0xf] ^ *pbVar1;
    }
    return;
}
```

Where does this key come from?

```
                    const_ek

00010ace 2b 7e 15       uint8_t[...  "+~",15h,16h,"(",AEh,D2h,A6h,AB
         16 28 ae
         d2 a6 ab ...
   00010ace 2b             uint8_t   '+'                          [0]
   00010acf 7e             uint8_t   '~'                          [1]
   00010ad0 15             uint8_t   15h                          [2]
   00010ad1 16             uint8_t   16h                          [3]
   00010ad2 28             uint8_t   '('                          [4]
   00010ad3 ae             uint8_t   AEh                          [5]
   00010ad4 d2             uint8_t   D2h                          [6]
   00010ad5 a6             uint8_t   A6h                          [7]
   00010ad6 ab             uint8_t   ABh                          [8]
   00010ad7 f7             uint8_t   F7h                          [9]
   00010ad8 15             uint8_t   15h                          [10]
   00010ad9 88             uint8_t   88h                          [11]
   00010ada 09             uint8_t   '\t'                         [12]
   00010adb cf             uint8_t   CFh                          [13]
   00010adc 4f             uint8_t   'O'                          [14]
   00010add 3c             uint8_t   '<'                          [15]
```


Appendix B — Cipher Example

The following diagram shows the values in the state array as the cipher progresses for a block length and a key length of 16 bytes each (i.e., $Nb = 4$ and $Nk = 4$).

```
Input  = 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34
Key    = 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c
```

The Round Key values are taken from the Key Expansion example in Appendix A.1.

Round Number	Start of Round	After SubBytes	After ShiftRows	After MixColumns	Round Key Value
input	32 88 31 e0 43 5a 31 37 f6 30 98 07 a8 8d a2 34				2b 28 ab 09 7e ae f7 cf 15 d2 15 4f 16 a6 88 3c
1	19 a0 9a e9 3d f4 c6 f8 e3 e2 8d 48 be 2b 2a 08	d4 e0 b8 1e 27 bf b4 41 11 98 5d 52 ae f1 e5 30	d4 e0 b8 1e bf b4 41 27 5d 52 11 98 30 ae f1 e5	04 e0 48 28 66 cb f8 06 81 19 d3 26 e5 9a 7a 4c	a0 88 23 2a fa 54 a3 6c fe 2c 39 76 17 b1 39 05

It uses the example AES key from the FIPS 197 standard...

The counter block initialization

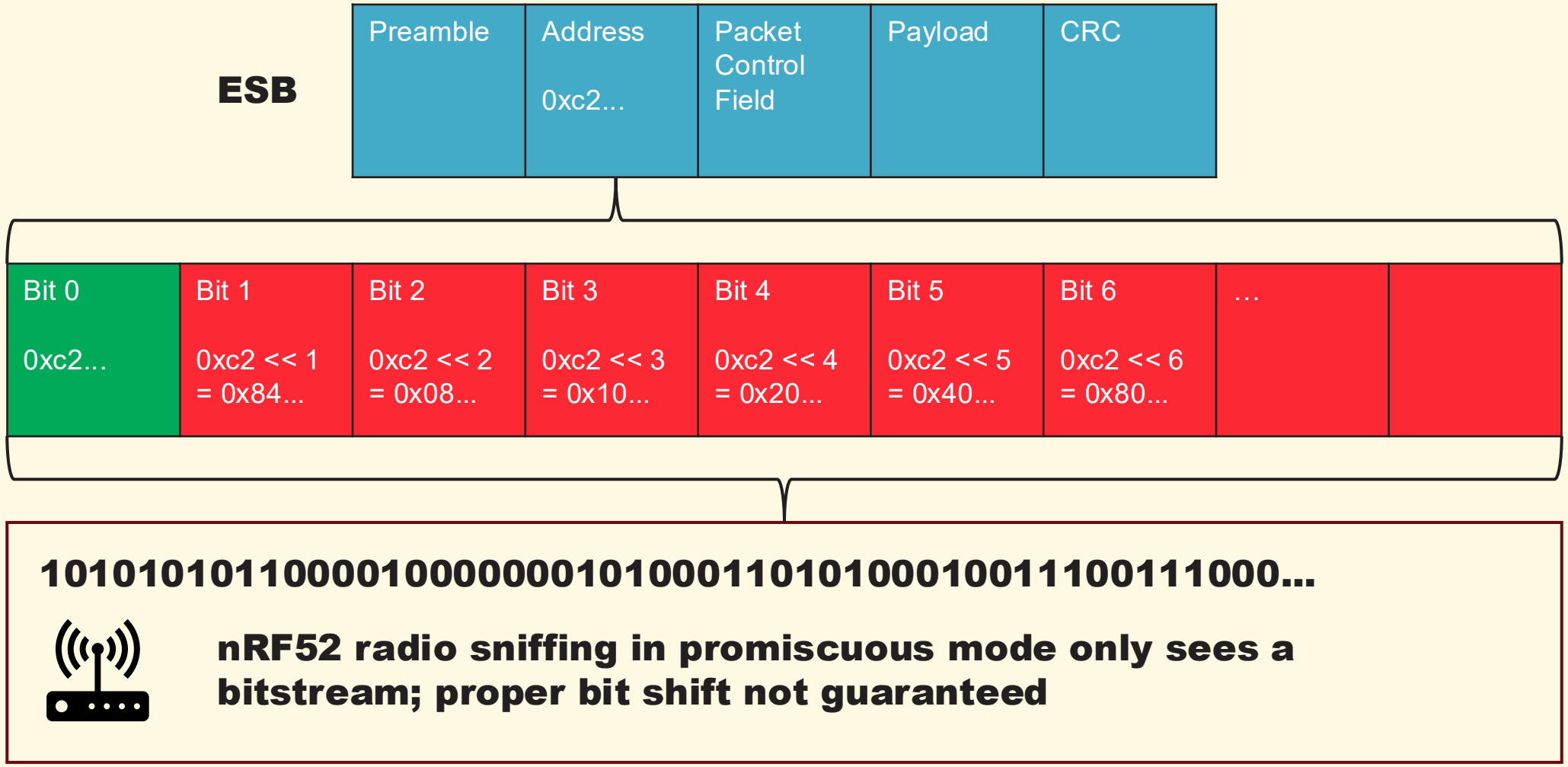
```
void zmk_24g_enc_update(uint8_t seq)

{
  // aes_ctx_pair is only used during pairing
  // aes_ctx is used for normal traffic
  memset(aes_ctx_pair + 6,0xaa,8);
  aes_ctx[0xe] = '\0';
  aes_ctx[0xf] = '\0';
  aes_ctx_pair[0xe] = '\0';
  aes_ctx_pair[0xf] = '\0’;
  // the ESB address is used for counter block
  memcpy(aes_ctx + 6,stored_sys_addr,8);
memcpy(aes_ctx_pair + 6,s_pairing_address,4);
  return;
}
```

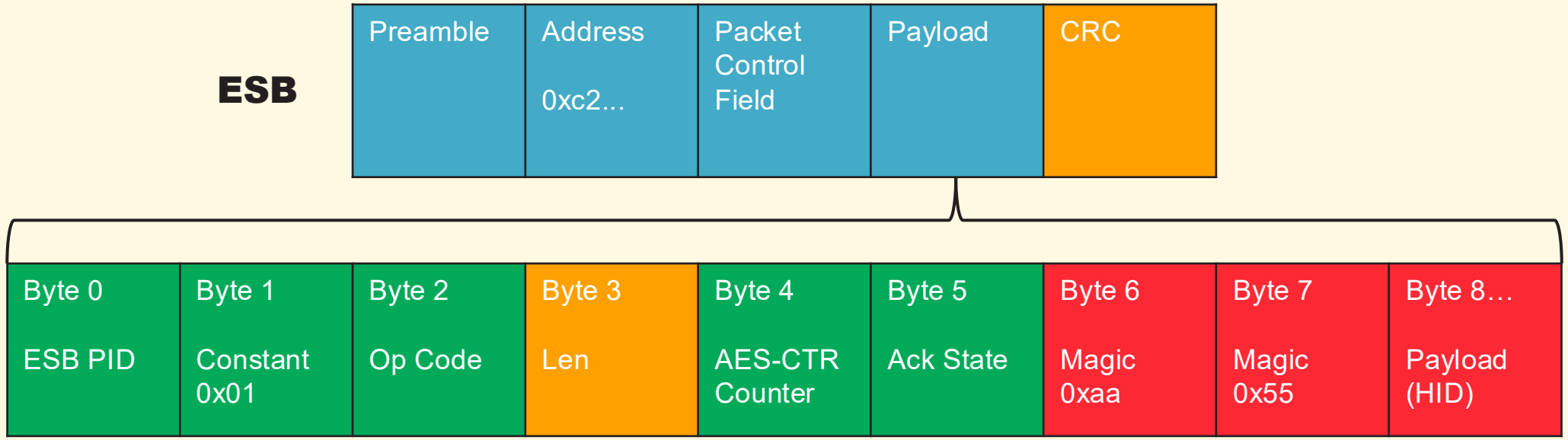
For normal traffic’s counter block:

- **Bytes 1-6 are a hard-coded string**
- **Bytes 7-14 are a per-pair link “secret”, first three sent in plaintext as ESB address**
- **Byte 15: Per-message counter**
- **Byte 16: Just (byte 15) * 2**

Hard to sniff the correct address



CRC gives an oracle!



10101010110000100000001010001101010001001110011100 0...

Computed CRC-16 MUST agree with rest of expected packet (using length)

Plan of attack

Sniff — Capture encrypted traffic in **promiscuous mode**

Slide — Test different bit shifts and **derive** the expected address, payload, CRC, etc.

Verify — Using the length from the payload, **compute** the CRC to confirm

Lock On — Select the **most-common** derived address.

Decrypt — Using an expected (incomplete) plaintext and the plaintext AES-CTR counter, probabilistically derive the **keystream**.

Magic 0xaa	Magic 0x55	HID Byte 0 Modifier Bits 0x00	HID Byte 1 Reserved 0x00	HID Byte 2 Key Press 1 ??	HID Byte 3 Key Press 2 ??	…	0x00	0x00	0x00

Demo

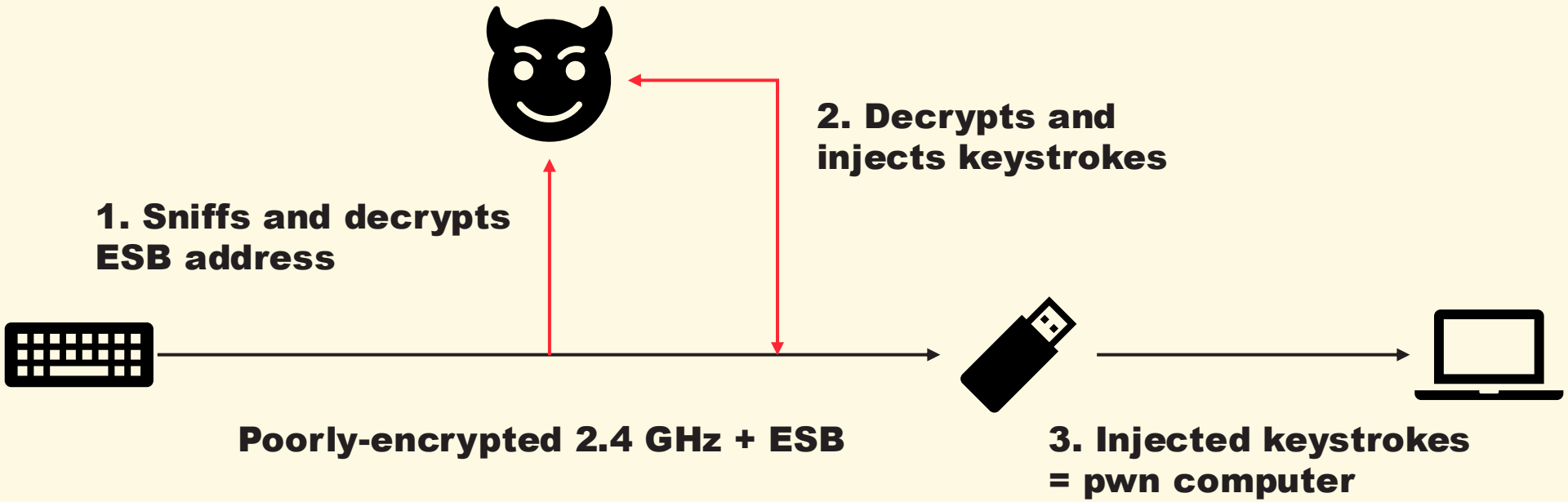


- **XIAO-nRF52840**
- **Arduino board based on Nordic nRF52840 radio**
- **Custom firmware to support Python scripting**

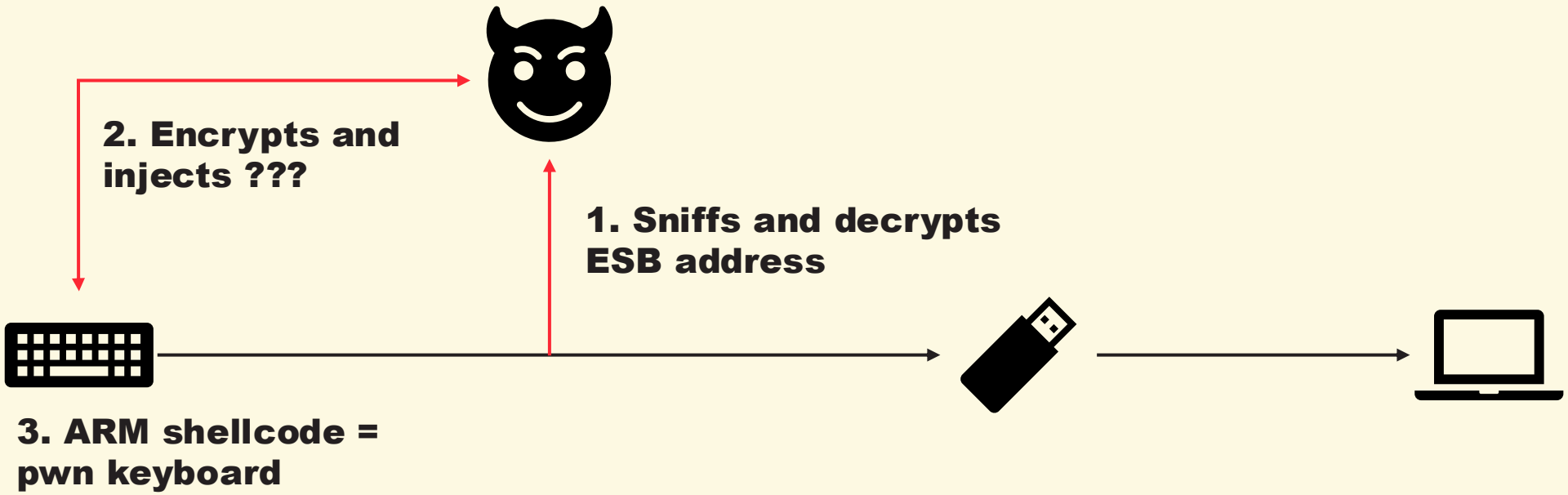
“But isn’t this just Mousejack++?”

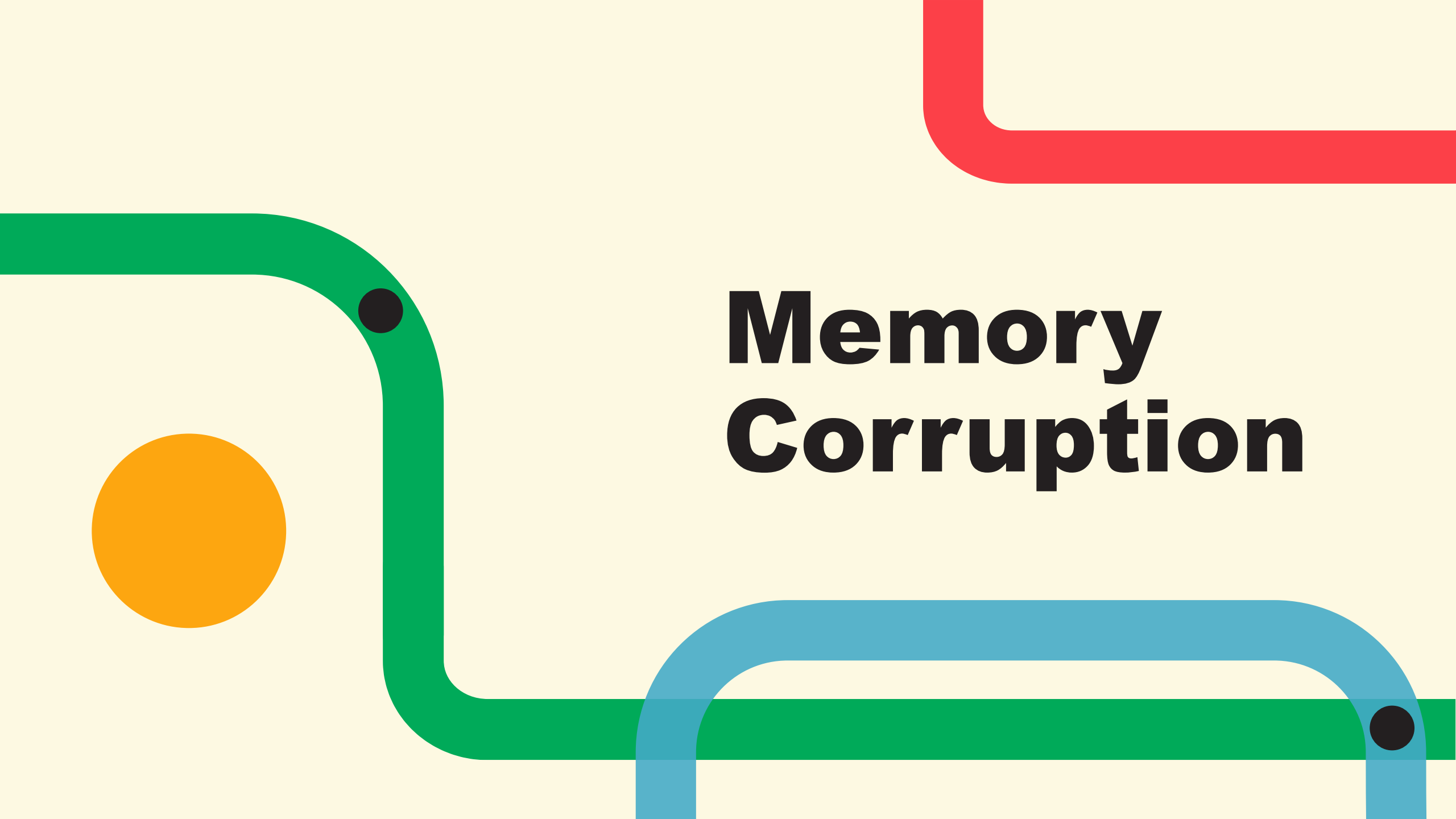
No crypto -> Weak crypto

You're right...


2. Decrypts and
injects keystrokes
1. Sniffs and decrypts
ESB address
Poorly-encrypted 2.4 GHz + ESB
3. Injected keystrokes
= pwn computer

...so let's try harder 😉




Memory
Corruption

Recap on VIA



A keyboard configuration protocol that rides in raw 32-byte HID packets with a simple opcode and data structure.

ID	Command	Effect
0x01	get_protocol_version	handshake
0x02/0x03/0x04	get/set keyboard value	uptime, layout options, matrix state…
0x05	dynamic_keymap_set_keycode	write one (layer,row,col) → keycode
…		

***Typically* restricted to USB cable inputs due to configuration powers…**

Proprietary mods strike again

Open-source QMK upstream: VIA only used over USB

```
void raw_hid_task(void) {
    uint8_t buffer[RAW_EPSIZE];
    while (receive_report(USB_ENDPOINT_OUT_RAW, buffer,
sizeof(buffer))) {
        raw_hid_receive(buffer, sizeof(buffer));
    }
}

void raw_hid_receive(uint8_t *data, uint8_t length) {
    uint8_t *command_id   = &(data[0]);
    uint8_t *command_data = &(data[1]);

    // begin VIA handler
    if (via_command_kb(data, length)) {
        return;
    }
// ...
```

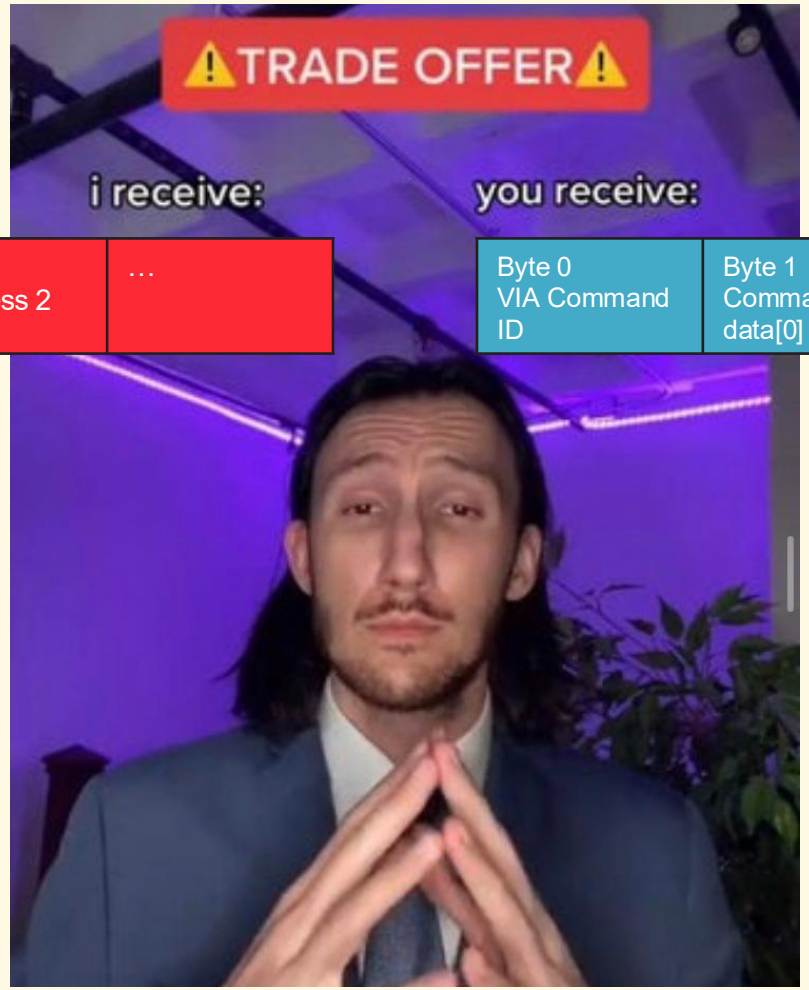
Proprietary ZMK mod: VIA allowed over 2.4 GHz

```
void zmk_24g_esb_rx(uint8_t *data,uint8_t len)
{
  // ...
  // begin VIA handler
  if (rx.cmd != 0xf6) {
    if (rx.cmd == 0xf7) {
      via_data_handler(rx.field_7.payload + 2,rx.len);
      active_count = 10;
      // ...
```

With crypto broken, I can do:

Decrypted HID input reports

Byte 0 Modifier Bits	Byte 1 Reserved	Byte 2 Key Press 1	Byte 3 Key Press 2	…



Encrypted raw HID / VIA command output reports

Byte 0 VIA Command ID	Byte 1 Command data[0]	Byte 2 Command data[1]	Byte 3 Command data[2]	…

ID	Command	Effect
0x01	get_protocol_version	handshake
0x02/0x03/0x04	get/set keyboard value	uptime, layout options, matrix state…
0x05	dynamic_keymap_set_keycode	write one (layer,row,col) → keycode
…		

The attack in detail



Capture encrypted keystroke frames

Build a table of counter-to-incomplete-keystream combinations

Send a forged (incomplete) 0x01 VIA command; response is known plaintext that is XORed to full keystream

Build a table of counter-to-complete-keystream combinations

Send forged (complete) VIA commands

Magic 0xaa	Magic 0x55	HID Byte 0 Modifier Bits 0x00	HID Byte 1 Reserved 0x00	HID Byte 2 Key Press 1 ??	HID Byte 3 Key Press 2 ??	…

Magic 0xaa	Magic 0x55	VIA command error 0xFF	0x00	…

Available VIA commands

At this point, there are several juicy-looking VIA commands:

- **dynamic_keymap_macro_set_buffer: inject keystrokes on host**
- **eeprom_reset: wipe settings**

...but this isn't code execution yet. Can we try harder?

qmk_firmware / quantum / via.h

Code | Blame | 191 lines (164 loc) · 6.81 KB

```
54  enum via_command_id {
55      id_get_protocol_version               = 0x01, // always 0x01
56      id_get_keyboard_value                 = 0x02,
57      id_set_keyboard_value                 = 0x03,
58      id_dynamic_keymap_get_keycode         = 0x04,
59      id_dynamic_keymap_set_keycode         = 0x05,
60      id_dynamic_keymap_reset               = 0x06,
61      id_custom_set_value                   = 0x07,
62      id_custom_get_value                   = 0x08,
63      id_custom_save                        = 0x09,
64      id_eeprom_reset                       = 0x0A,
65      id_bootloader_jump                    = 0x0B,
66      id_dynamic_keymap_macro_get_count     = 0x0C,
67      id_dynamic_keymap_macro_get_buffer_size = 0x0D,
68      id_dynamic_keymap_macro_get_buffer    = 0x0E,
69      id_dynamic_keymap_macro_set_buffer    = 0x0F,
70      id_dynamic_keymap_macro_reset         = 0x10,
71      id_dynamic_keymap_get_layer_count     = 0x11,
72      id_dynamic_keymap_get_buffer          = 0x12,
73      id_dynamic_keymap_set_buffer          = 0x13,
74      id_dynamic_keymap_get_encoder         = 0x14,
75      id_dynamic_keymap_set_encoder         = 0x15,
76      id_unhandled                          = 0xFF,
77  };
```

A comment is not code...

```
case id_dynamic_keymap_get_buffer: {
    uint16_t offset = (command_data[0] << 8) | command_data[1];
    uint16_t size = command_data[2]; // size <= 28
    dynamic_keymap_get_buffer(offset, size, &command_data[3]);
    break;
}

void dynamic_keymap_get_buffer(uint16_t offset, uint16_t size, uint8_t *data) {
    // 768 bytes, far larger than than destination buffer of 28
    uint16_t dynamic_keymap_eeprom_size = DYNAMIC_KEYMAP_LAYER_COUNT * MATRIX_ROWS * MATRIX_COLS * 2;
    uint16_t source = (DYNAMIC_KEYMAP_EEPROM_ADDR + offset);
    uint8_t *target = data;
    for (uint16_t i = 0; i < size; i++) {
        if (offset + i < dynamic_keymap_eeprom_size) {
            *target = eeprom_read_byte(source);
        } else {
            *target = 0x00;
        }
        source++;
        target++;
    }
}
```

A simple exploit trigger chain

Stage

Store payload bytes legitimately in keymap memory. Do this multiple times as needed.

dynamic_keymap_set_buffer 0x13	offset_1 0x00	offset_2 0x24	size 0x1C	Data

Verify

Read payload bytes legitimately from keymap memory to confirm it was staged properly.

dynamic_keymap_get_buffer 0x12	offset_1 0x00	offset_2 0x24	size 0x1c

Pwn

Overwrite a return address in the stack with staged bytes.

dynamic_keymap_get_buffer 0x12	offset_1 0x00	offset_2 0x00	size 0x28

ARM Shellcode

Most Memory Mitigations Missing

Zephyr offers multiple mitigation options like CONFIG_STACK_CANARIES or CONFIG_HW_STACK_PROTECTION, but when running on a microcontroller with no MMU, most memory mitigations are impractical or completely impossible.

Mitigation	Status	Result
Stack canary	Off	No early termination
ASLR	None	Can use hardcoded addresses
Execute-never (XN)	On	Cannot jump straight to shellcode; ROP needed

Time to ROP and Roll

Return address

Location	Purpose
sp+0x2c	Saved Link Register (LR) return address for via_worker overwritten by keymap[36..39]
sp+0x28	Saved r4 overwritten by keymap[32..35]
...	
sp+0x08	Copy starts here (&command_data[3])
sp+0x04	buf[0]

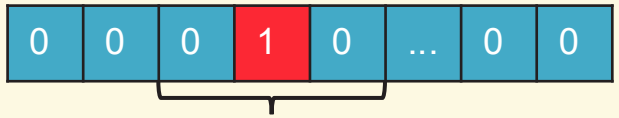
ROP Gadgets

- **Useful arbitrary write:**
  - **pop {r0, r1, r2, r3, pc}**
  - **str r2, [r3]**
- **Disable Memory Protection Unit (and thus XN) at hard-coded 0xE000ED94:**
  - **pop {r0-r3,pc} r2 = 0, r3 = 0xE000ED94**
  - **str r2, [r3] MPU->CTRL = 0**
- **Cortex-M operates in ARM Thumb mode**
  - **Need to ensure PC always has LSB of 1 or it will HardFault**

From RGB to RCE to... RGB?

With arbitrary shellcode achieved, we can make RCE physically visible by manipulating the GPIO-mapped memory for the keyboard's LEDs!

0x5000050C: Output-clear register (pin LOW) for GPIO port 0



Bits 28-30 correspond to RGB for the keyboard' status indicator LED



LED turns red! Quickly cycle between combinations of these 3 bits to get a nice gradient effect.

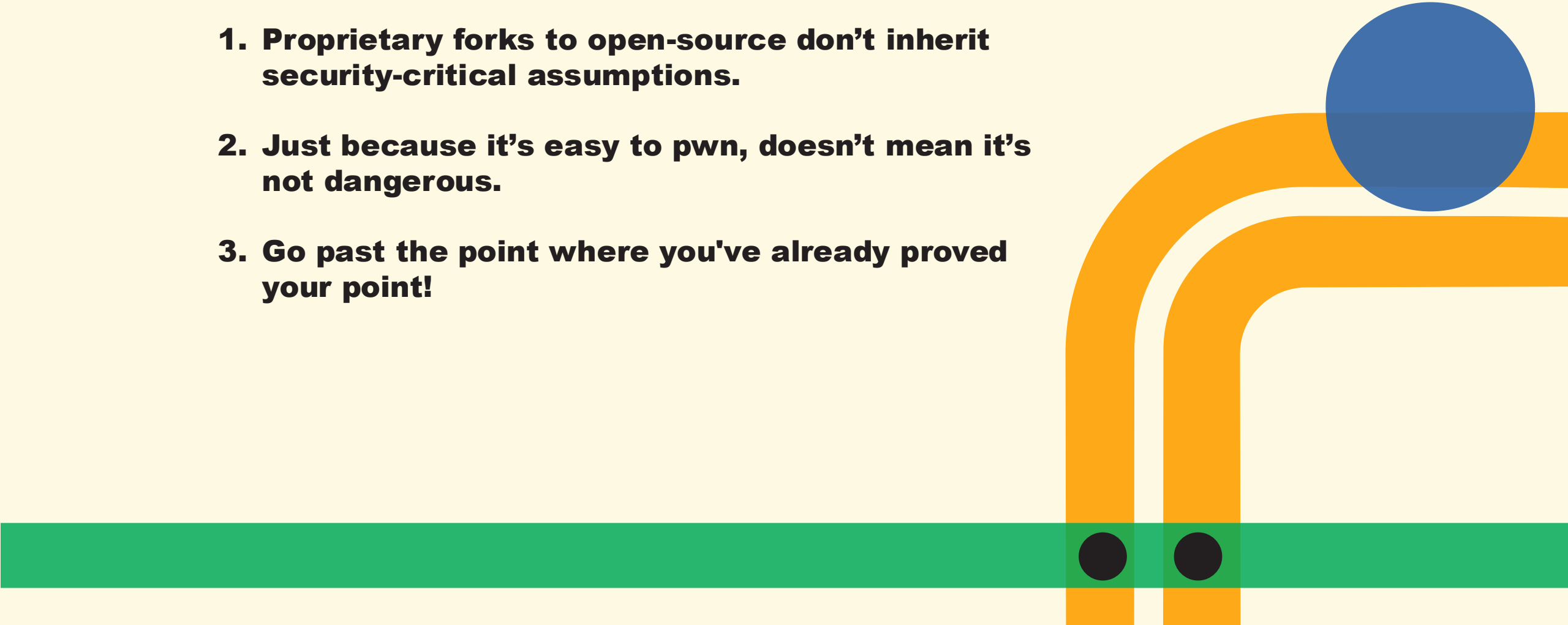
Demo

Takeaways

1. **Proprietary forks to open-source don’t inherit security-critical assumptions.**

2. **Just because it’s easy to pwn, doesn’t mean it’s not dangerous.**

3. **Go past the point where you've already proved your point!**



Thank you

Eugene Lim

spaceraccoon.dev

